

APEX Semester Planning Agent

Project report

Live demo: <https://technion-semester-planner-agent.vercel.app>

Project Summary

APEX is a Technion semester-planning agent. The student describes their situation in natural language, and the system builds a semester plan with courses, timetable, exams, risks, and explanations.

The project has two parts. The offline part is a data pipeline that prepares reliable course data. The online part is an agent that chooses tools, checks plans, repairs them, and answers the student.

Supported Use Cases

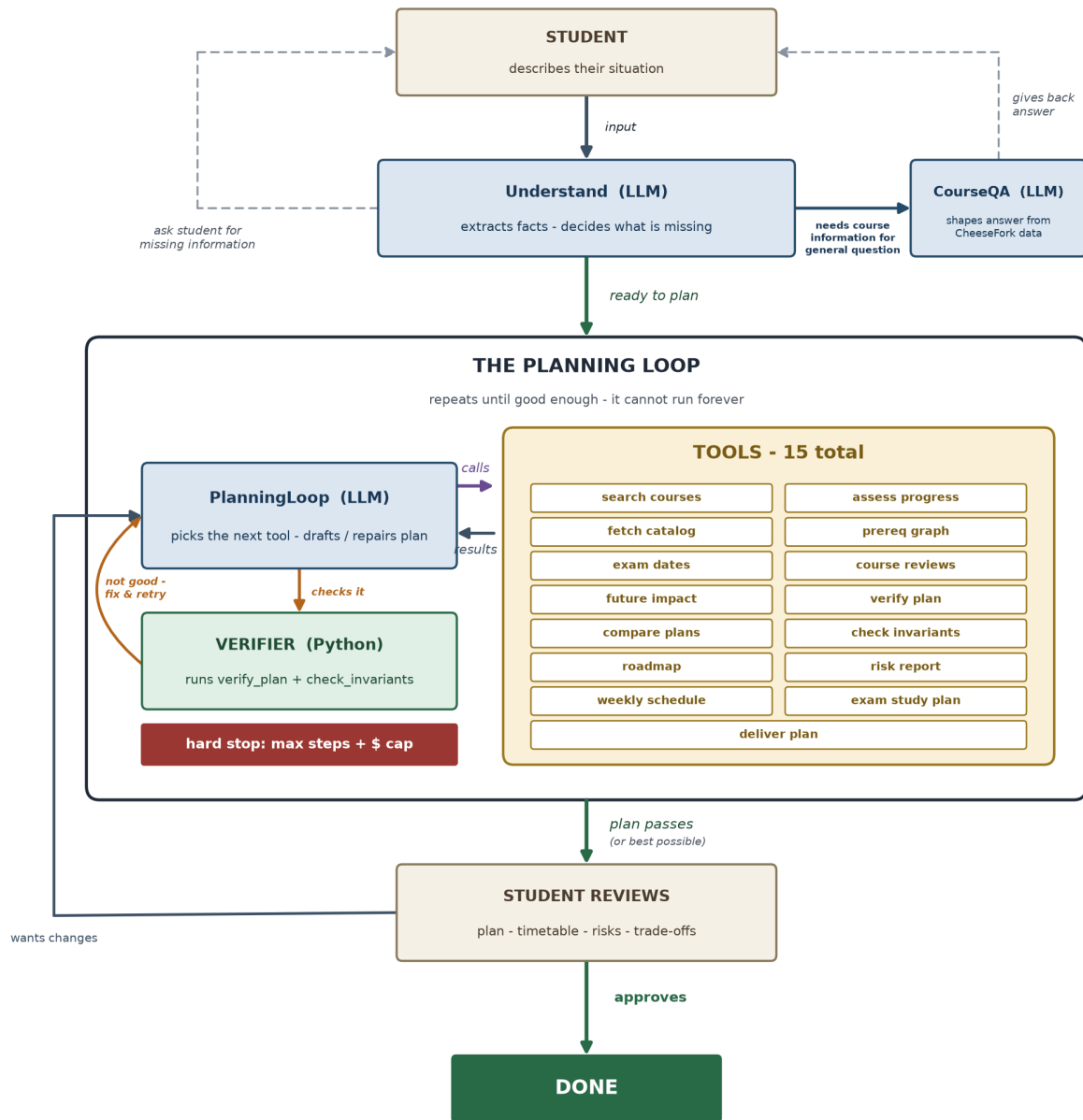
- Build a semester plan from Hebrew or English free text.
- Respect blocked weekdays, pace preference, passed courses, and failed courses.
- Prioritize failed retakes when they block future courses.
- Suggest grade-improvement retakes only as a proposal, never as a silent addition.
- Answer course questions using real CheeseFork review data.
- Revise a plan on follow-up, such as adding, removing, or swapping a course.
- Parse an uploaded transcript PDF into courses and grades.
- Optionally restore sessions with Supabase, using a hashed student email.

Architecture

The default runtime is a bounded ReAct-style loop. The model does not produce a plan in one shot. It receives a set of tools, chooses which tool to call, reads the tool result, and decides the next step.

Module	Role
Understand	Extracts structured student state from the conversation.
PlanningLoop	Chooses tools, drafts plans, verifies plans, and repairs when needed.
CourseQA	Answers focused course questions without running a full planner.

Semester Planning Agent - Model Architecture



Tools

The agent has exactly 15 model-callable tools. Three diagnostics are also injected automatically every planning turn to reduce cost and avoid repeated calls: progress assessment, catalog context, and prerequisite graph context.

Tool area	Examples
Course search and catalog	<code>search_courses</code> , <code>fetch_catalog</code> , <code>query_prereq_graph</code>
Reviews and grades	<code>summarize_cheesefork</code> , grade-improvement analysis from technion-histograms
Plan checks	<code>verify_plan</code> , <code>check_invariants</code> , <code>compare_plans</code>
Schedule and exams	<code>weekly_schedule_analyzer</code> , <code>fetch_exam_dates</code> , <code>exam_study_planner</code>
Progress and risk	<code>assess_progress</code> , <code>simulate_future_impact</code> , <code>roadmap_to_graduation</code> , <code>risk_report</code>
Final answer	<code>deliver_plan</code>

Data Sources

Source	Used for
Technion course API	Catalog, requirements, prerequisites, schedules, sections, and exam dates.
Official track diagrams	Correct semester placement for mandatory courses.
CheeseFork	Student reviews, difficulty, satisfaction, and load.
technion-histograms	Historical course averages for grade-improvement reasoning.
Student input	Passed courses, failed courses, grades, and transcript PDF.
Supabase	Optional session persistence.

Why There Is No Large RAG Layer

The main planning data is structured. Course numbers, prerequisites, credits, section times, exam dates, and passed courses need exact checks. For this reason, the project uses typed tools rather than vector retrieval for the core planner.

A small policy RAG tool could be added later for academic regulations or administrative documents. It was not added to this version because it would not improve the core scheduling and verification task.

Prompt Size and Call Efficiency

The planning system prompt is long because it embeds the full eligible-course list (difficulty, satisfaction, exam date, mandatory status) for the semester, not because of unrelated padding. Every field in that list is read by name by a specific planning rule - for example, exam dates are compared across candidates to avoid scheduling two exams close together, and difficulty and satisfaction ratings are compared when choosing between electives. Fetching this data per course through tool calls instead would not reduce it: the same data would move into tool-result history and still be resent on later turns, while adding more model calls per plan - working against "avoid unnecessary LLM calls," not toward it.

Prompt size was not independently trimmed for cost because it is not the cost driver here. Input tokens are priced well below output tokens (\$0.25 vs \$2.00 per million), and this was measured, not assumed. Output length and step count - not prompt length - are what actually determine the cost of a planning turn.

Required Endpoints

Endpoint	Purpose
GET /api/team_info	Returns the required team metadata for the assignment.
GET /api/agent_info	Returns description, purpose, prompt template, examples, and steps.
GET /api/model_architecture	Returns the architecture diagram as PNG.
POST /api/execute	Runs a one-shot prompt and returns status , error , response , and steps .

Current Limits

- The system does not register students to courses.
- The system does not access private Technion accounts.
- The data bundle is a snapshot, with a final live offering check before delivery.
- If a fully clean plan is not found, the agent reports the best honest plan and names the remaining issue.